



Universidade de Passo Fundo - UPF

Instituto de Tecnologia - ITEC

Ubiquitous Computing: Clouds, IoT e Smart Environments

Comparação de APIs em Go, Python e Node.js Desenvolvimento, Docker e Testes de Desempenho

Igor Zanette¹, Marco A. Santolin², Marcos P. de Medeiros³

¹Instituto de Tecnologia – Universidade de Passo Fundo (UPF)
Ubiquitous Computing: Clouds, IoT e Smart Environments

198862@upf.br¹, 198769@upf.br², 151994@upf.br³

Abstract. *This work compares three user APIs implemented in Go (Gin), Python (FastAPI), and Node.js (Express). All applications expose CRUD operations over isolated PostgreSQL schemas, run in Docker containers, and are accessed through a Rust-based API Gateway. JMeter load tests measured latency, throughput, and errors. Results show Go achieved highest throughput, Node.js maintained zero errors with high latency, and Python balanced performance and development simplicity.*

Resumo. *Este trabalho compara três APIs REST implementadas em Go (Gin), Python (FastAPI) e Node.js (Express). Todas executam operações CRUD sobre schemas isolados no PostgreSQL, empacotadas em contêineres Docker e acessadas via API Gateway em Rust. Testes de carga com JMeter mediram latência, vazão e erros. Go atingiu maior vazão, Node.js manteve zero erros com alta latência, e Python equilibrou desempenho e simplicidade de desenvolvimento.*

1. Introdução

Sistemas de computação ubíqua exigem serviços de backend confiáveis e escaláveis. APIs RESTful são a interface de comunicação predominante entre dispositivos, gateways e serviços de nuvem. A escolha da linguagem e do framework para implementar essas APIs influencia diretamente o desempenho, a escalabilidade e a experiência de desenvolvimento. Este trabalho compara três APIs de usuários desenvolvidas em Go [Donovan and Kernighan 2016] (utilizando Gin [Vyas and Contributors 2014]), Python [Lutz 2013] (com FastAPI [Ramírez 2018]) e Node.js [Tilkov and Vinoski 2010] (com Express). O objetivo é avaliar como diferentes ecossistemas se comportam quando submetidos a cenários de carga e estresse.

2. Implementação

O sistema é composto por três serviços de API independentes — em Go, Python e Node.js — cada um executando operações CRUD sobre a entidade *users*. Cada serviço utiliza um schema próprio no PostgreSQL (*go_app*, *python_app* e *node_app*). Todas as APIs

são empacotadas em contêineres Docker, e o arquivo `docker-compose.yml` sobre **duas instâncias de cada API**. Um API Gateway em Rust [Klabnik and Nichols 2019] (Actix-web) realiza balanceamento de carga round-robin entre as instâncias.

A API em Go [Donovan and Kernighan 2016] utiliza Gin [Vyas and Contributors 2014] para rotas REST e Gorm para ORM, com criptografia bcrypt. Python [Lutz 2013] emprega FastAPI [Ramírez 2018] com programação assíncrona, SQLAlchemy e driver asyncpg. Node.js [Tilkov and Vinoski 2010] utiliza Express e a biblioteca pg para acesso ao banco. Detalhes completos da implementação estão disponíveis no repositório do GitHub.

3. Testes e resultados

Os testes de desempenho foram realizados com Apache JMeter em três rodadas: **Rodada 1** com 100 threads × 2 repetições (200 requisições), **Rodada 2** com 500 threads × 5 repetições (2.500 requisições), e **Rodada 3** com 1.000 threads × 10 repetições (10.000 requisições). As métricas incluíram tempo de resposta, throughput, erros e latência. Todos os testes foram executados na mesma máquina: Intel Core i7-14700KF (20 cores/28 threads), 32 GB DDR5 6000 MHz, SSD M.2 2 TB.

3.1. Resultados consolidados

A Tabela 1 apresenta o resumo consolidado das três rodadas de testes para cada API.

Table 1. Resultados consolidados das três rodadas de teste					
API	Rodada	Tempo (ms)	Throughput (req/s)	Erro (%)	Latência (ms)
Go	1	7.0	20.11	0.0	7.3
Go	2	7.33	246.26	0.0	7.39
Go	3	204.83	434.38	41.75	174.12
Node.js	1	6.67	20.14	0.0	7.1
Node.js	2	365.33	111.55	0.0	365.79
Node.js	3	1265.5	111.65	0.0	1265.85
Python	1	1.83	20.19	0.0	2.16
Python	2	0.5	250.11	0.0	1.13
Python	3	663.0	192.69	0.01	663.32

3.2. Análise dos resultados

Na **Rodada 1** (200 requisições), todas as APIs apresentaram desempenho satisfatório com taxa de erro zero. Python surpreendeu com o menor tempo de resposta (1.83 ms). Na **Rodada 2** (2.500 requisições), Go manteve baixa latência (7.33 ms) e alto throughput (246.26 req/s), enquanto Node.js apresentou degradação significativa (365.33 ms). Na **Rodada 3** (10.000 requisições), Go registrou 41.75% de erros devido à limitação `max_connections = 100` do PostgreSQL, insuficiente para 1.000 threads simultâneas. Apesar disso, manteve o maior throughput (434.38 req/s). Node.js atingiu latência de 1265.85 ms sem erros, indicando enfileiramento de requisições. Python apresentou comportamento intermediário (663.32 ms, 0.01% erros).

As Figuras 1, 2 e 3 ilustram a progressão das métricas ao longo das rodadas.

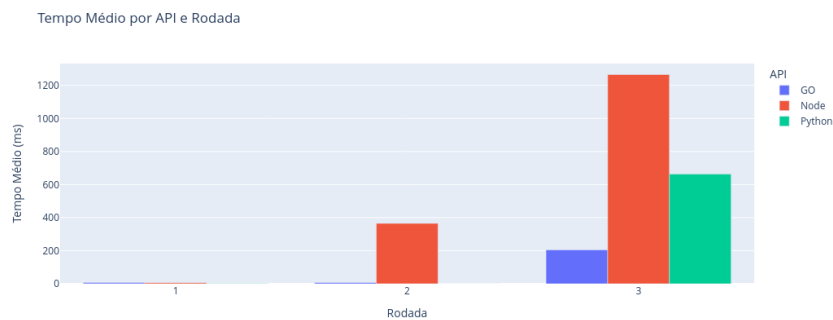


Figure 1. Tempo médio de resposta por API

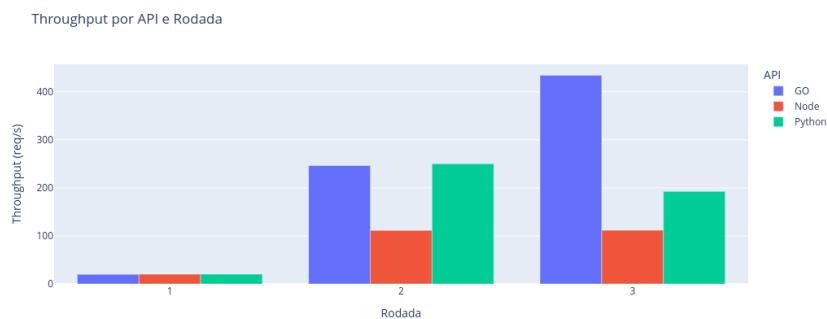


Figure 2. Throughput por API

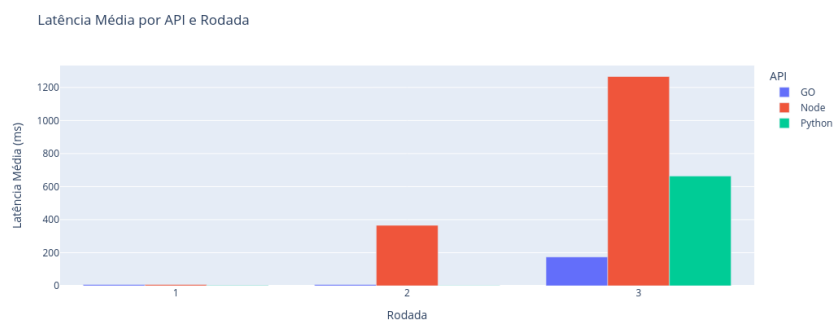


Figure 3. Latência média por API

A Tabela 2 apresenta o relatório consolidado de erros por API (todas as rodadas).

Table 2. Relatório de erros por API			
API	Total Requisições	Total Erros	Taxa de Erro (%)
Go	76200	25052	32.88
Node.js	76200	0	0.0
Python	76200	4	0.01

3.3. Discussão

Go [Donovan and Kernighan 2016] demonstrou a melhor escalabilidade, com throughput superior sob alta carga. A taxa de erro de 41.75% na rodada 3 resulta da limitação `max_connections = 100` do PostgreSQL, não de falha da linguagem. O comportamento *fail-fast* de Go é adequado para sistemas distribuídos com circuit breakers. Node.js [Tilkov and Vinoski 2010] manteve taxa de erro zero enfileirando requisições, mas sofreu degradação severa de latência (µs), possivelmente devido ao *event loop* single-threaded. Python [Lutz 2013] apresentou comportamento equilibrado, com FastAPI [Ramírez 2018] e SQLAlchemy gerenciando bem o pool de conexões. Para produção, recomenda-se ajustar `max_connections` proporcionalmente à carga esperada.

4. Conclusão

Este trabalho comparou três implementações de APIs RESTful em ambientes containerizados com balanceamento de carga. Go destacou-se em throughput e escalabilidade, Node.js ofereceu estabilidade com degradação de latência, e Python equilibrou facilidade de desenvolvimento com desempenho intermediário. A escolha entre as tecnologias deve considerar requisitos de desempenho, familiaridade da equipe e configuração adequada de infraestrutura.

References

- [Donovan and Kernighan 2016] Donovan, A. A. A. and Kernighan, B. W. (2016). *The Go Programming Language*. Addison-Wesley Professional, 1st edition.
- [Klabnik and Nichols 2019] Klabnik, S. and Nichols, C. (2019). *The Rust Programming Language*. No Starch Press, 1st edition. Official Rust Book, supported by the Rust Project Developers.
- [Lutz 2013] Lutz, M. (2013). *Learning Python*. O'Reilly Media, 5th edition.
- [Ramírez 2018] Ramírez, S. (2018). Fastapi: Modern, fast (high-performance) web framework for building apis with python 3.6+. <https://fastapi.tiangolo.com>. Acessado em 2025.
- [Tilkov and Vinoski 2010] Tilkov, S. and Vinoski, S. (2010). *Node.js: Using JavaScript to Build High-Performance Network Programs*. IEEE Internet Computing. Foundational publication describing Node.js concepts.
- [Vyas and Contributors 2014] Vyas, M. and Contributors, G.-G. (2014). Gin: Http web framework written in go (golang). <https://gin-gonic.com>. Acessado em 2025.